Desarrollo e implementación con tecnologías libres de un dispositivo portátil para evaluar el confort térmico, lumínico y acústico en interiores

José Ramón Hernández Aguilar

2024-12-25

Tabla de contenidos

Portada

1 Introducción al confort en espacios espacios interiores y el desarrollo del DTHIS-C

En este capítulo se examina la importancia del confort en espacios interiores conforme al ANSI/ASHRAE Standard 55, que establece las condiciones térmicas necesarias para el bienestar de los ocupantes. Se incluyen modelos adaptativos de confort, que explican cómo las personas ajustan sus expectativas de acuerdo con el clima y contexto, aportando claves para el diseño de entornos confortables y eficientes.

Se presenta también el DTHIS-C (Dispositivo de Temperatura, Humedad, Iluminación y Sonido de Campaña), un equipo portátil que mide las siguiente variables físicas:

- Temperatura del aire a diferentes alturas
- Temperatura radiante
- Humedad relativa
- Velocidad del viento
- Concentración de CO2
- Luminancia
- Deslumbramiento
- Niveles de sonido

Destacando la importancia de su elaboración para el desarrollo de espacios interiores basados en criterios de confort térmico, acústico y lumínico.

1.1 Importancia del confort en espacios interiores

1.2 Modelos de confort adaptativos

1.3 DTHIS-C

2 Confort y calidad del aire en espacios interiores

En este capítulo se aborda el contexto del confort en espacios cerrados, incluyendo los aspectos térmico, acústico, lumínico y la calidad del aire. Se define el concepto de confort térmico, destacando las variables físicas que lo determinan. Además, se identifican y describen los principales fenómenos asociados con el disconfort térmico, acústico y lumínico. Finalmente, se presentan los aspectos fundamentales relacionados con el confort acústico, lumínico y la calidad del aire.

2.1 Confort térmico

El confort térmico se define como una condición mental que refleja la satisfacción con el entorno térmico, de acuerdo con el estándar ASHRAE-55 (2021).

Según Djongyang, Tchinda, y Njomo (2010), el confort térmico se alcanza cuando la temperatura corporal se mantiene dentro de rangos estrechos, la humedad en la piel es baja y el esfuerzo fisiológico para regular la temperatura es mínimo. Además, el confort no solo depende de factores ambientales, sino también de diferentes aspectos, como el tipo de vestimenta, el tipo de actividad física, cambiar de postura o ubicación, abrir una ventana, etc.

2.2 Variables físicas del confort térmico

Macpherson (1962) identificó seis factores determinantes que afectan la sensación térmica en los espacios cerrados. Entre ellos, cuatro son variables físicas: temperatura del aire, velocidad del aire, humedad relativa y temperatura media radiante; y dos son variables personales: el aislamiento térmico de la ropa y el nivel de actividad física, también conocido como tasa metabólica. Estos factores constituyen la base para el análisis y predicción del confort térmico.

Acorde a ASHRAE-55 (2021), dichas variables se definen de la siguiente manera:

1. **Temperatura del aire**: La temperatura medida en un punto específico del aire.
2. **Velocidad del aire**: La magnitud del movimiento del aire en un punto determinado, sin considerar su dirección.
3. **Humedad**: Representa el contenido de humedad en el aire y se expresa mediante varias variables termodinámicas, como presión de vapor, temperatura de punto de rocío, temperatura de bulbo húmedo, relación de humedad y humedad relativa. Su promedio espacial y temporal se calcula de manera similar al de la temperatura del aire.
4. **Temperatura media radiante (t_r)**: La temperatura de un recinto negro y uniforme que intercambia con el ocupante la misma cantidad de calor por radiación que el entorno real. Este valor único aplica a todo el cuerpo y considera tanto la temperatura radiante media de onda larga (t_{rlw}) como la de onda corta (t_{rsw}).
5. **Aislamiento térmico de la ropa (I_{cl})**: La resistencia al intercambio de calor sensible proporcionada por un conjunto de ropa, expresada en unidades de "clo". *Nota informativa*: Esta definición incluye la transferencia de calor de todo el cuerpo, incluyendo las áreas no cubiertas, como la cabeza y las manos.

6. **Tasa metabólica** (*met*): La tasa a la que se transforma la energía química en calor y trabajo mecánico debido a las actividades metabólicas de un individuo. Se mide por unidad de área de superficie de la piel y se expresa en "met", equivalente a 58.2 W/m2, que corresponde a la energía generada por unidad de superficie de piel de una persona promedio en reposo.

2.3 Confort acústico

El confort acústico se define como "un estado de satisfacción con las condiciones acústicas", representa el bienestar y la percepción positiva de un entorno en términos sonoros. Sin embargo, este confort es altamente subjetivo, ya que fuentes de ruido con características físicas similares pueden percibirse de manera distinta dependiendo de la persona. Antoniadou y Papadopoulos (2017) Azar et al. (2020)

2.4 Variables físicas del confort acústico

Azar et al. (2020) nombra dos tipos principales de ruido en los edificios:

- **Ruido estructural (por impacto)**: Generado por impactos físicos o vibraciones sobre un elemento de la construcción.
- **Ruido aéreo**: Transmitido a través del aire.

El **nivel de presión sonora**, definido como la vibración del aire que constituye el sonido Berglund (1999), es uno de los principales factores acústicos que influyen en el confort. Un nivel neutral de presión sonora se encuentra en un rango de 30 a 40 dB Antoniadou y Papadopoulos (2017). Para predecir el confort frente al ruido de impacto, se utiliza típicamente el nivel máximo de presión sonora (L_{max}), mientras que para el ruido aéreo

se emplea el nivel de presión sonora equivalente durante un periodo de tiempo determinado (L_{eq}). Azar et al. (2020)

Asimismo Berglund (1999), enumera ciertos factores acústicos que impactan el confort acústico:

1. **Frecuencia del ruido**: Su unidad de medida son los Hertz (Hz) y se refiere al número de vibraciones por segundo del aire en el cual el sonido es propagado. El rango de frecuencia audible normalmente se encuentra en un rango de 20-20,000 Hz.
2. **Fuente del ruido**: Se mencionan diferentes fuentes, las cuales pueden ser industrial, de transporte, construcción y doméstico.
3. **Duración del ruido**
4. **Variación en el tiempo**: Existen variaciones de la presión sonora que suelen causar mayor disconfort auditivo. Por ejemplo, los ruidos que varían periódicamente para crear una sensación de pulsación pueden ser más molestos que el ruido continuo, las variaciones de aproximadamente 4 por segundo podrían ser más perturbadoras.

2.5 Confort lumínico

El confort lumínico se refiere a la percepción de bienestar visual en un espacio, permitiendo que las personas realicen sus actividades cotidianas de manera efectiva y cómoda bajo condiciones de iluminación adecuadas. Vásquez et al. (2019)

2.6 Variables físicas del confort lumínico

Según se describe en la norma ISO EN 12464-1 Standard (2011), la evaluación del confort lumínico puede lograrse mediante la determinación de: la

luminancia y la iluminancia.

1. **Luminancia**: Su unidad de medida es la candela por metro cuadrado (cd/m^2) y se define como la densidad de la intensidad luminosa en una dirección específica, por unidad de área proyectada en un punto determinado de una superficie real o imaginaria. CIE (2020b)

   Matemáticamente, se expresa como:

   $L_v = \frac{I_v}{A \cdot \cos(\alpha)}$

   donde (L_v) es la luminancia, (I_v) representa la intensidad luminosa en la dirección especificada, (A) es el área considerada, y (α) es el ángulo entre la normal a la superficie en el punto especificado y la dirección considerada.

   La luminancia cuantifica cuán brillante aparece una superficie cuando se observa desde una dirección específica, teniendo en cuenta tanto la intensidad de la luz emitida o reflejada como la geometría de la superficie.

2. **Iluminancia**: Su unidad de medida es el lux (lx) y se define como la densidad del flujo luminoso incidente por unidad de área en un punto de una superficie real o imaginaria. CIE (2020a)

   Matemáticamente, se expresa como:

   $E_v = \frac{\Phi_v}{A}$

   donde (E_v) es la iluminancia, (Φ_v) representa el flujo luminoso incidente, y (A) es el área sobre la cual incide el flujo luminoso.

   La iluminancia cuantifica la cantidad de luz que llega a una superficie específica, considerando tanto la intensidad de la luz como el área sobre la cual se distribuye.

2.7 Calidad del aire

De acuerdo con el estándar ASHRAE-62 (2001), se considera que el aire interior tiene una calidad aceptable cuando "no contiene contaminantes conocidos en concentraciones nocivas según lo establecido por las autoridades competentes y la mayoría sustancial de las personas expuestas (80% o más) no expresa insatisfacción".

2.8 Variables físicas de la calidad del aire

La calidad del aire interior se evalúa monitoreando la **concentración de CO2**, comparándola con los niveles exteriores (300-500 ppm). ASHRAE-62 (2001) recomienda mantener concentraciones interiores entre 1000 y 1200 ppm para garantizar un ambiente adecuado y regular los sistemas de ventilación. Sin embargo, concentraciones superiores a 5000 ppm son peligrosas, ya que indican una ausencia total de aire fresco.

3 Diseño y construcción del DTHIS-C

En este capítulo se presenta una descripción detallada del DTHIS-C, incluyendo sus componentes y especificaciones técnicas. Además, se proporcionan los diagramas de conexión y los códigos necesarios para la configuración de cada sensor, facilitando tanto la reproducción completa del DTHIS-C como la implementación individual de sus sensores.

3.1 Especificaciones del DTHIS-C

En la Tabla **??** se muestran los sensores y componentes electrónicos que conforman al DTHIS-C.

Tabla 3.1: Sensores y componentes electrónicos que consitutyen al DTHIS-C.

Sensor	**Variable**	**Comunicación**	**Microcontrolador**	**Acondicionador de señal**
Termopar tipo T	Temperatura ambiente	ICSP	Arduino UNO WiFi Rev2	PWFusion MAX31856 SEN-30007

Sensor	Variable	Comunicación	Microcontrolador	Acondicionador de señal
TPF1/E-20 PT1000	Temperatura radiante	ICSP	Arduino UNO WiFi Rev2	Adafruit PT1000 RTD-MAX31865
Wind Sensor Rev P6	Velocidad del viento	ADC	Arduino UNO WiFi Rev2	
SCD30 Sensirion	CO2 y humedad relativa	I2C	Raspberry Pi 4 Model B	
5MP OV5647 Wide Angle Fisheye Camera	Luminancia	CSI	Raspberry Pi 4 Model B	
Micrófono ambiental USB	Sonido	USB	Raspberry Pi 4 Model B	

3.2 Diseño del DTHIS-C

La arquitectura del DTHIS-C se ha fundamentado en el uso combinado de la Raspberry Pi 4 y el Arduino Uno WiFi Rev2, microcontroladores seleccionados por las siguientes características y ventajas:

1. **Versatilidad y capacidad de procesamiento:**
   La Raspberry Pi 4, programada mediante Python, proporciona una plataforma robusta para el procesamiento avanzado de datos y la ejecución de algoritmos complejos. Su flexibilidad, junto con la facilidad de integración de diversas funcionalidades, permite adaptar el

sistema a requerimientos específicos y optimizar el rendimiento según las demandas del entorno.

2. **Conectividad y eficiencia en la adquisición de datos:**
   Ambos dispositivos cuentan con conectividad a Internet, lo que posibilita la transmisión en tiempo real de datos hacia una base de datos remota alojada en el servidor ThingsBoard. Además, el Arduino Uno WiFi Rev2 destaca por su eficiencia en la gestión de operaciones de entrada/salida, garantizando una respuesta rápida y precisa en la adquisición y control de los sensores.

3. **Compatibilidad y ecosistema abierto:**
   La adopción de tecnologías abiertas facilita la integración y reproducibilidad del sistema. La amplia comunidad de desarrolladores que respalda tanto la Raspberry Pi como el Arduino asegura soporte continuo, actualizaciones regulares y una extensa variedad de recursos y librerías, lo que se traduce en mayor flexibilidad para adaptar el sistema a diversas necesidades y entornos.

4. **Accesibilidad:**
   Frente a equipos especializados, la utilización de estos microcontroladores constituye una solución rentable sin comprometer el rendimiento ni la fiabilidad del sistema. Esta ventaja económica facilita la replicación y el escalado del proyecto en distintos contextos.

A continuación, se detallan las especificaciones técnicas y el código implementado para cada sensor.

3.2.1 Arduino UNO WiFi Rev2

El Arduino UNO WiFi Rev2 es una placa de microcontrolador diseñada para proyectos de Internet de las Cosas (IoT), integrando conectividad WiFi y Bluetooth mediante el módulo u-blox NINA-W102 y un sensor IMU de 6 ejes (LSM6DS3TR) para detección de movimientos. Equipada con el microcontrolador ATmega4809, ofrece una óptima capacidad de memoria,

y mantiene compatibilidad con el formato estándar UNO, facilitando su uso con shields existentes. Su diseño eficiente y versátil lo hace ideal para la adquisición de datos de sensores y la comunicación inalámbrica en tiempo real, como se requiere en el DTHIS-C. Arduino (2023)

En la Tabla **??** se muestran sus especificaciones.

Tabla 3.2: Especificaciones del Arduino UNO WiFi Rev2.

Parámetro	**Detalle**
Microcontrolador	ATmega4809
Memoria flash	48 KB
SRAM	6,144 Bytes
EEPROM	256 Bytes
Conectividad Wi-Fi	Sí, u-blox NINA-W102 (2.4 GHz)
Bluetooth	Sí, Bluetooth Low Energy
Sensor IMU	Sí, LSM6DS3TR (6 ejes)
Pines digitales I/O	14 (5 PWM)
Pines analógicos	6
Voltaje de operación	5V
Consumo máximo total	200 mA
Dimensiones	68.6 mm x 53.4 mm

El siguiente código constituye el programa principal del Arduino UNO WiFi Rev2 en el DTHIS-C, encargado de gestionar la conexión WiFi, la comunicación con ThingsBoard mediante el protocolo MQTT, y el envío de datos provenientes de sensores como termopares, temperatura radiante y velocidad del viento:

```
#include <WiFiNINA.h>
#include <PubSubClient.h>

// Definiciones para la red WiFi y ThingsBoard
```

```
#define WIFI_AP "IER"
#define WIFI_PASSWORD "acadier2014"
#define TOKEN "bXQq1eepFYJV3nldvLTV"
#define THINGSBOARD_SERVER "tb.ier.unam.mx"
#define MQTT_PORT 1883

// Inicialización de las librerías y objetos
WiFiClient espClient;
PubSubClient mqttClient(espClient);

/**
 * Configuración inicial del dispositivo.
 * Configura la conexión WiFi y el cliente MQTT para ThingsBoard.
 */
void setup() {
  WiFi.begin(WIFI_AP, WIFI_PASSWORD);  // Conectar a la red WiFi
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);  // Espera hasta conectarse
  }

  mqttClient.setServer(THINGSBOARD_SERVER, MQTT_PORT);
  connectToThingsBoard();  // Conectar a ThingsBoard
}

/**
 * Ciclo principal del programa.
 * Mantiene las conexiones y realiza las tareas periódicas.
 */
void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    reconnectWiFi();  // Reconecta WiFi si es necesario
  }
```

```
  if (!mqttClient.connected()) {
    reconnectThingsBoard();  // Reconecta MQTT si es necesario
  }

  // Obtener las mediciones de los termopares (temperaturas)
  float* temperatures = ThermocoupleTemp();
  sendTC(temperatures);    // Enviar temperaturas a ThingsBoard

  // Obtener la temperatura del TPF1
  float radiant = TPF1Temp();
  sendRT(radiant);  // Enviar temperatura radiante a ThingsBoard

  // Obtener la velocidad promedio del viento
  float* windspeed = WindSensor();
  sendWS(windspeed);      // Enviar velocidad del viento a ThingsBoard

  mqttClient.loop();
  delay(2000);  // Espera 2 segundos antes de la siguiente iteración
}

/**
 * Conecta a la red WiFi.
 */
void connectWiFi() {
  WiFi.begin(WIFI_AP, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);  // Espera hasta conectarse
  }
}

/**
 * Reconecta a la red WiFi si es necesario.
 */
```

```
void reconnectWiFi() {
  while (WiFi.status() != WL_CONNECTED) {
    WiFi.begin(WIFI_AP, WIFI_PASSWORD);
    delay(500);
  }
}

/**
 * Reconecta a ThingsBoard si es necesario.
 */
void reconnectThingsBoard() {
  while (!mqttClient.connected()) {
    connectToThingsBoard();
    delay(5000);  // Espera antes de reintentar
  }
}

/**
 * Conecta a ThingsBoard usando el token de autenticación.
 */
bool connectToThingsBoard() {
  return mqttClient.connect("ArduinoClient", TOKEN, NULL);
}

/**
 * Envía los datos de temperatura.
 */
void sendTC(float* temperatures) {
  String topic = "v1/devices/me/telemetry";
  String payload = "{\"T1\":" + String(temperatures[0]) +
                   ", \"T2\":" + String(temperatures[1]) +
                   ", \"T3\":" + String(temperatures[2]) +
                   ", \"T4\":" + String(temperatures[3]) + "}";
```

```
  mqttClient.publish(topic.c_str(), payload.c_str());
}

/**
 * Envía los datos de temperatura radiante.
 */
void sendRT(float radiant) {
  String topic = "v1/devices/me/telemetry";
  String payload = "{\"TR\":" + String(radiant) + "}";
  mqttClient.publish(topic.c_str(), payload.c_str());
}

/**
 * Envía los datos de velocidad del viento.
 */
void sendWS(float* windspeed) {
  String topic = "v1/devices/me/telemetry";
  String payload = "{\"WS\":" + String(windspeed[0]) +
                   ", \"TEMP\":" + String(windspeed[1]) + "}";
  mqttClient.publish(topic.c_str(), payload.c_str());
}
```

3.2.1.1 Termopar Tipo T

En la construcción del DTHIS se emplea un termopar tipo T, un sensor de temperatura compuesto por la unión de cobre y cobre-níquel. Su uso es ideal para entornos con humedad.

A continuación, se presentan sus especificaciones según OMEGA (s. f.):

Rangos de Temperatura
- **Grado de termopar:** -200 a 350 °C (-328 a 662 °F).
- **Grado de extensión:** -60 a 100 °C (-76 a 212 °F).

Límites de Error
- **Estándar:**
- ±1.0 °C o ±0.75% por encima de 0 °C.
- ±1.0 °C o ±1.5% por debajo de 0 °C.
- **Especial:**
- ±0.5 °C o ±0.4%.

3.2.1.2 PWFusion MAX31856 SEN-30007

El PWFusion MAX31856 SEN-30007 es un shield de termopares de cuatro canales diseñado para integrarse con la plataforma Arduino. Este dispositivo se conecta directamente a la placa de desarrollo. El módulo acondiciona la señal proveniente de termopares, realizando el procesamiento necesario para adaptar y amplificar la señal eléctrica, la cual se caracteriza por su baja amplitud.

Acorde a PWFusion (s. f.) sus específicaciones son las siguientes:

- Conversión de señal analógica a digital con resolución de 19 bits.
- Interfaz ICSP de 4 hilos.
- Rango de voltaje de alimentación: 3.3 V a 5.0 V.

Igualmente se proporciona el código de Arduino empleado:

```
#include <PWFusion_MAX31856.h>

// Definición de los pines chip select para cada termopar
uint8_t tcChipSelects[] = {10, 9, 8, 7};
#define NUM_THERMOCOUPLES   (sizeof(tcChipSelects) / sizeof(uint8_t))

// Coeficientes de calibración para cada termopar
const float m[NUM_THERMOCOUPLES] = {1.0450, 1.0602, 1.0814, 1.0554};  // Pendientes
const float b[NUM_THERMOCOUPLES] = {-2.4327, -2.8632, -4.4316, -3.9348};  // Intersecciones
```

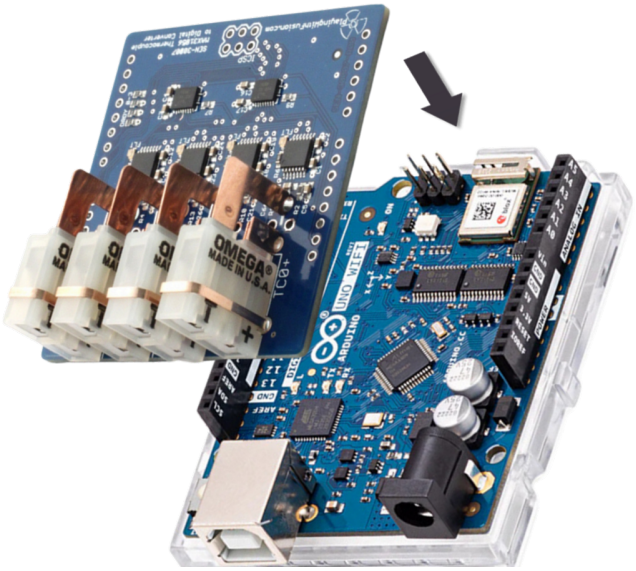

Figura 3.1: Diagrama de conexión para el shield MAX31856 SEN-30007.

```
// Se crea un objeto MAX31856 para cada termopar
MAX31856 thermocouples[NUM_THERMOCOUPLES];

float* ThermocoupleTemp() {
  static float tc_temps[NUM_THERMOCOUPLES];
  static bool initialized = false;

  // Inicializa los sensores solo la primera vez que se llama a la función
  if (!initialized) {
    delay(1000);  // Espera para la estabilización de los sensores
    for (int i = 0; i < NUM_THERMOCOUPLES; i++) {
      thermocouples[i].begin(tcChipSelects[i]);
      thermocouples[i].config(T_TYPE, CUTOFF_60HZ, AVG_SEL_4SAMP, CMODE_OFF);
    }
    initialized = true;
  }

  // Inicia la medición en cada sensor
  for (int i = 0; i < NUM_THERMOCOUPLES; i++) {
    thermocouples[i].startOneShotMeasurement();
  }

  // Espera el tiempo necesario para que la conversión se complete
  delay(180);

  // Actualiza la muestra en cada sensor
  for (int i = 0; i < NUM_THERMOCOUPLES; i++) {
    thermocouples[i].sample();
  }

  // Obtiene la temperatura ambiente de cada sensor y aplica la calibración
  for (int i = 0; i < NUM_THERMOCOUPLES; i++) {
    if (thermocouples[i].getStatus() != 0) {
```

```
      // En caso de error, se asigna NaN (Not a Number)
      tc_temps[i] = NAN;
    } else {
      float raw_temperature = thermocouples[i].getTemperature();
      tc_temps[i] = m[i] * raw_temperature + b[i];  // Aplica la calibración
    }
  }

  return tc_temps;
}
```

3.2.1.3 Fuehler Systeme TPF1/E-20 PT1000

Se empleó el TPF1/E-20 PT1000, diseñado para medir la temperatura radiante en un rango de temperaturas de -30 °C a +75 °C. Su sonda de temperatura, basada en la tecnología PT1000, se puede montar como un péndulo libremente suspendido, lo que permite obtener mediciones precisas de la temperatura de sensación térmica en entornos donde la convección natural y la estratificación del aire pueden influir en los resultados. FuehlerSysteme (s. f.)

Especificaciones

- Tipo de circuito: Conexión de 2 hilos.
- Corriente de medición: Aproximadamente 1 mA.
- Conexión eléctrica: Extremos pelados con terminales.
- Cable: Cable de PVC (2×0,25 mm2, temperatura máxima +105 °C) con extremos de cable núcleo disponibles en diferentes longitudes.
- Resistencia de fuga: Mayor a 100 MΩ a +20 °C (500 V DC).
- Material del globo: Aluminio (negro).
- Dimensiones del globo: Diámetro de 70 mm.

3.2.1.4 Adafruit PT1000 RTD-MAX31865

El PT1000 RTD-MAX31865 es un convertidor de resistencia a digital.

Adafruit (s. f.) describe las siguientes características:

- Realiza la conversión de la resistencia de sensores RTD de platino a un valor digital de forma sencilla.
- Soporte con RTD de platino con valores de 100 Ω a 1 kΩ (a 0 °C), abarcando desde PT100 hasta PT1000.
- Compatible con configuraciones de 2, 3 y 4 hilos para la conexión de sensores.
- Incorpora una interfaz ICSP, facilitando su integración en sistemas basados en microcontroladores.
- Cuenta con un ADC de 15 bits que proporciona una resolución nominal de temperatura de 0.03125 °C (valor variable debido a la no linealidad del RTD) y una precisión total de hasta 0.5 °C (0.05% de la escala completa) en todas las condiciones operativas.
- Tiempo máximo de conversión es de 21 ms.

A continuación se muestra el código de Arduino:

```
#include <Adafruit_MAX31865.h>

// Usamos software SPI: CS, DI, DO, CLK
Adafruit_MAX31865 thermo = Adafruit_MAX31865(10, 11, 12, 13);
// Para hardware SPI solo se pasa el pin CS
// Adafruit_MAX31865 thermo = Adafruit_MAX31865(10);

#define RREF      4300.0  // Resistor de referencia (usa 430.0 para PT100 o 4300.0 para PT10
#define RNOMINAL  1000.0  // Resistencia nominal a 0°C (100.0 para PT100, 1000.0 para PT1000

// Función que realiza la medición y retorna la temperatura radiante en grados Celsius.
float TPF1Temp() {
  // Inicializa el sensor solo la primera vez que se llama a la función.
```

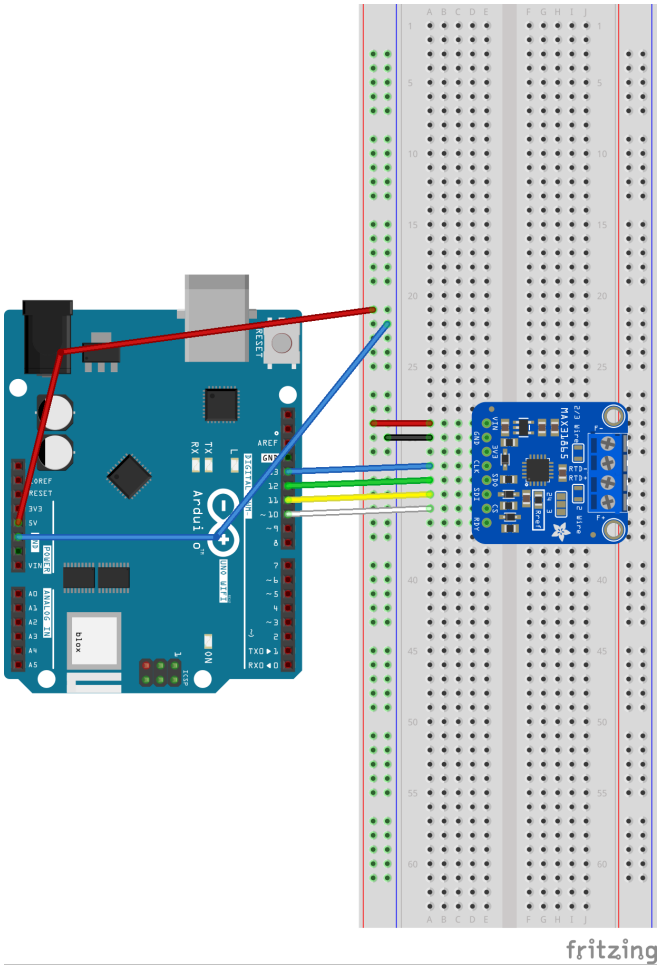

Figura 3.2: Diagrama de conexión para el PT1000 RTD-MAX31865.

```
  static bool initialized = false;
  if (!initialized) {
    thermo.begin(MAX31865_2WIRE);  // Inicialización del MAX31865
    initialized = true;
  }

  // Lectura del valor RTD
  uint16_t rtd = thermo.readRTD();

  // Calcula la razón y la resistencia (opcional si se requiere para otros cálculos)
  float ratio = rtd / 32768.0;
  float resistance = RREF * ratio;

  // Calcula la temperatura utilizando el método de la librería
  float raw_temp = thermo.temperature(RNOMINAL, RREF);

  // Aplica la ecuación de calibración: 1.0582 * temp + (-1.5553)
  float radiant_temp = 1.0582 * raw_temp - 1.5553;

  return radiant_temp;
}
```

3.2.1.5 Modern Device Wind Sensor Rev. P6

El sensor de viento Rev. P6 es un anemómetro de hilo caliente. Incorpora un potenciómetro de alta precisión para facilitar la calibración (realizada en fábrica) y utilizar termistores de coeficiente de temperatura positivo, que aseguran mediciones más estables incluso ante variaciones de temperatura ambiente. Requiere una fuente de alimentación de 9 a 12 V (idealmente 12 V) para garantizar el adecuado calentamiento de los termistores y evitar la saturación en condiciones de viento intenso. Integra un sensor de temperatura ambiental con salida escalada a 3.3 V y una señal de viento

ajustada a un máximo de 3.3 V mediante resistencias de alta precisión. ModernDevice (s. f.)

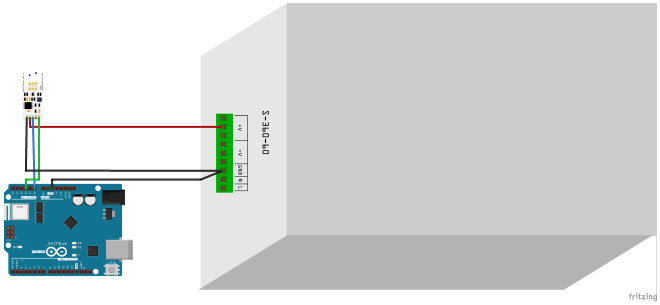

Figura 3.3: Diagrama de conexión para el Wind Sensor Rev. P6.

Especificaciones

- Voltaje de alimentación: 10–12 V.
- Corriente: Aproximadamente 40 mA.
- Velocidades de viento medidas: 0–150 mph.
- Compensación de temperatura ambiente.

```
#include <math.h>

const int OutPin  = A0;   // Pin analógico para el sensor de viento ("OUT")
const int TempPin = A2;   // Pin analógico para el sensor de temperatura ("TI

// Función que lee velocidad de viento y temperatura, y retorna un array de c
// ws[0] -> velocidad de viento (m/s)
// ws[1] -> temperatura (°C)
float* WindSensor() {
  static float ws[2];

  // Lectura del sensor de viento
  int windADunits = analogRead(OutPin);
```

```
  float voltage = (float)windADunits * 5.0 / 1024.0;

  // Aplicar la corrección al voltaje
  float correctedVoltage = voltage - 1.1621;

  // Lectura del sensor de temperatura
  int tempRawAD = analogRead(TempPin);
  float tempC = ((((float)tempRawAD * 5.0) / 1024.0) - 0.400) / 0.0195;

  // Cálculo de la velocidad de viento en m/s usando la ecuación empírica:
  // WS = 26.3431 * (correctedVoltage)^(1.4273) * (tempC)^(-0.7631)
  float windSpeed = 26.3431 * pow(correctedVoltage, 1.4273) * pow(tempC, -0.7631);

  ws[0] = windSpeed;
  ws[1] = tempC;

  return ws;
}
```

3.2.2 Raspberry Pi 4 Model B

El Raspberry Pi 4 Model B es una computadora de placa única de alto rendimiento, diseñada para tareas de procesamiento avanzado y conectividad eficiente en un formato compacto. Equipada con un procesador de 64 bits de cuatro núcleos y opciones de RAM de hasta 8 GB, ofrece soporte para salida de video dual 4K y conectividad avanzada mediante Wi-Fi de doble banda, Bluetooth 5.0, Ethernet Gigabit y puertos USB 3.0, con capacidad para Power over Ethernet (PoE) mediante un accesorio adicional. Su diseño versátil y eficiente la hace ideal para la integración y gestión de datos en el sistema DTHIS-C. Raspberry (2019)

En la Tabla **??** se muestran sus especificaciones.

Tabla 3.3: Especificaciones del Raspberry Pi 4 Model B.

Parámetro	**Detalle**
Procesador	Broadcom BCM2711, 4 núcleos Cortex-A72, 1.5 GHz
Memoria RAM	8 GB (LPDDR4-3200)
Conectividad Wi-Fi	Sí, dual-band 2.4 GHz y 5 GHz, 802.11ac
Bluetooth	Sí, Bluetooth 5.0
Puertos USB	2 USB 3.0, 2 USB 2.0
Salida de video	2 micro HDMI, soporte dual 4K 60 fps
Ethernet	Gigabit
Almacenamiento	Tarjeta microSD
Voltaje de operación	5V
Consumo energético	Eficiente, sin ventilador (~3A recomendado)
Dimensiones	85.6 mm x 56.5 mm x 17 mm

El siguiente código constituye el programa principal del Raspberry Pi 4 Model B en el DTHIS-C, encargado de gestionar la conexión al servidor ThingsBoard mediante el protocolo MQTT y el envío de datos provenientes de sensores como el SCD30 (CO2, temperatura y humedad relativa). Escrito en Python, utiliza la librería `paho-mqtt` para la comunicación y está diseñado para operar de manera continua:

```
import paho.mqtt.client as mqtt
import json
import time
from thingsboard_credentials import thingsboard_data
from scd30 import scd30_measurements
from sound import sound_data

# Obtiene los datos de conexión para ThingsBoard
unique_id, token, thingsboard_host = thingsboard_data()
```

```
# Diccionario para almacenar las lecturas del sensor
sensor_data = {'CO2': 0, 'T': 0, 'HR': 0}
# Diccionario para datos de sonido
# micro_data = {'RMS': 0, 'dBmax': 0, 'dBmin': 0}

# Configura el cliente MQTT
client = mqtt.Client(unique_id, False)
client.username_pw_set(token, password=None)
client.connect(thingsboard_host, 1883, 60, '')
client.loop_start()

try:
    while True:
        # Obtiene las mediciones del sensor SCD30
        CO2, T, HR = scd30_measurements()

        # Actualiza el diccionario con las lecturas actuales
        sensor_data['CO2'] = CO2
        sensor_data['T'] = T
        sensor_data['HR'] = HR

        # Publica los datos del SCD30 en ThingsBoard
        client.publish('v1/devices/me/telemetry', json.dumps(sensor_data), 1)

        # Obtiene los datos de sonido
        # micro_data = sound_data()

        # Publica los datos de sonido en ThingsBoard
        # client.publish('v1/devices/me/telemetry', json.dumps(micro_data), 1)

        time.sleep(2)  # Ajusta el intervalo de tiempo entre lecturas

except KeyboardInterrupt:
```

```
    pass

# Detiene el loop del cliente MQTT y desconecta del servidor
client.loop_stop()
client.disconnect()
```

3.2.2.1 Sensirion SCD30

El SCD30 es un sensor de CO2 basado en tecnología de detección por infrarrojos. Incorpora un sensor de humedad y temperatura. Cuenta con un diseño de doble canal. Además, ofrece una precisión de ±(30 ppm + 3% de la lectura) en un rango de 400 a 10,000 ppm. Sensirion (s. f.)

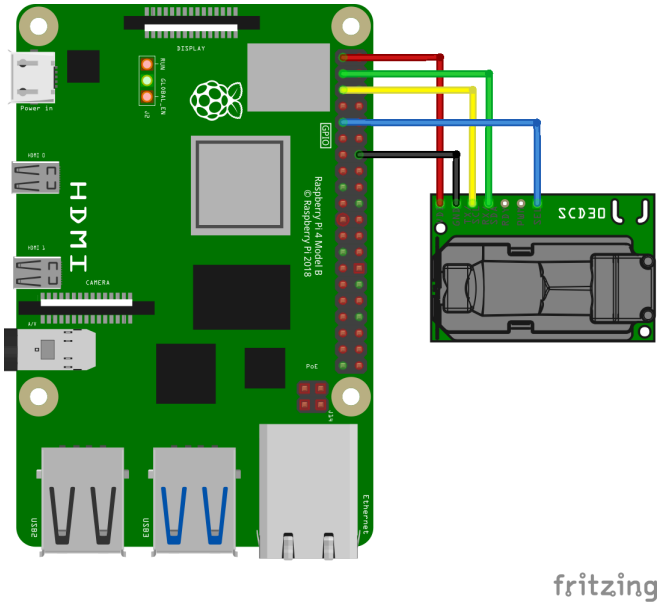

Figura 3.4: Diagrama de conexión para el SCD30.

Especificaciones

CO2

- Rango de medición: 400 – 10,000 ppm
- Precisión: ±30 ppm ±3% del valor medido
- Tiempo de respuesta (63%): 20 s
- Estabilidad térmica: 2.5 ppm/°C

Humedad

- Precisión típica: ±3 %RH
- Rango de operación: 0 – 95 %RH
- Tiempo de respuesta (63%): 8 s
- Certificación de calibración: Calibrado de fábrica
- Temperatura
- Precisión típica: ±0.4 °C
- Tiempo de respuesta (63%): 10 s

Características generales

- Voltaje de alimentación: 3.3 – 5.5 V
- Corriente promedio: 19 mA
- Corriente máxima: 75 mA
- Rango de temperatura de operación: 0 – 50 °C
- Interfaces: I²C, ModBus, PWM
- Dimensiones: 35 × 23 × 7 mm

A continuación se muestra el código de Python empleado para su incorpación al DTHIS-C:

```
import time
import board
import busio
from scd30_i2c import SCD30

def scd30_measurements():
```

```
    """
    Función para obtener las lecturas de CO2, Temperatura y Humedad Relativa

    Retorna:
        tuple: (CO2, T, HR) donde
            CO2: concentración de CO2 en ppm
            T: temperatura en grados Celsius
            HR: humedad relativa en porcentaje
    """
    # Inicializa el sensor SCD30
    scd30 = SCD30()
    # Espera para asegurarse de que el sensor esté listo
    time.sleep(2)

    # Verifica si los datos están listos para ser leídos
    if scd30.get_data_ready():
        # Lee las mediciones del sensor
        CO2, T, HR = scd30.read_measurement()
        # Redondea cada valor a dos decimales
        CO2 = round(CO2, 2)
        T = round(T, 2)
        HR = round(HR, 2)
        return CO2, T, HR
```

3.2.2.2 Arducam 5MP OV5647 Ultra Wide Angle

La cámara Arducam Ultra Wide Angle cuenta con un objetivo ojo de pez M12 que ofrece un campo de visión horizontal de 220°. Arducam (s. f.)

Especificaciones

- Sensor: Omnivision OV5647
- Tamaño del sensor: 1/4 (OV5647)

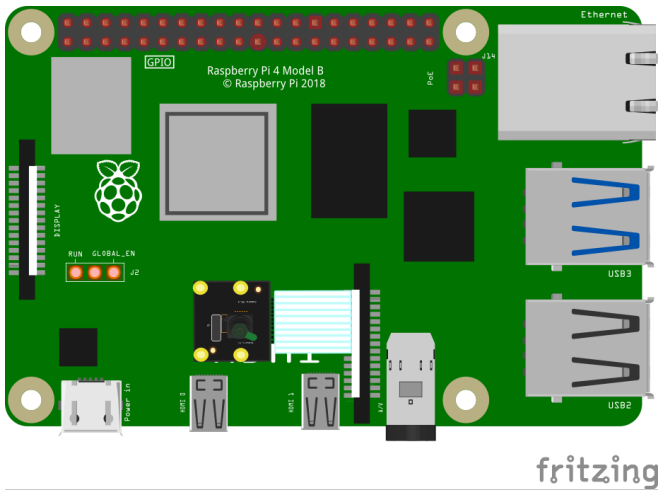

Figura 3.5: Diagrama de conexión para la cámara OV5647.

- Resolución: 5 MP (2592×1944)
- Video:
  - 1080p a 30 fps
  - 720p a 60 fps
  - 480p a 90 fps
- Sensibilidad IR: Filtro IR-cut integrado (solo luz visible)
- Campo de visión horizontal (HFOV): 220°
- Distancia focal efectiva: 0.76 mm
- Distancia focal equivalente en 35 mm: 8.2 mm
- Tipo de enfoque: Manual
- Montura del objetivo: M12
- Dimensiones: 36 mm × 36 mm
- Corriente pico: 300 mA

A continuación, se muestra el código escrito en Bash/Shell que accede a la cámara y, posteriormente, genera los mapas de luminancia:

```
#!/bin/bash

DATE=$(date +"%Y-%m-%d_%H%M")
IMG_DIR="/home/hdeza/Luminance/images"
HDR_IMG="/home/hdeza/Luminance/HDR_images"
MAPS_DIR="/home/hdeza/Luminance/maps"
OUTPUT_DIR="/home/hdeza/Luminance"

#Loop through shutter speeds and capture images
SHUTTERS=(100 500 1000 5000 10000 50000 100000 500000 1000000 2000000)

for i in "${!SHUTTERS[@]}"; do
  rpicam-still --raw -n --gain 1 -t 500 --shutter "${SHUTTERS[$i]}" -o "$IMG_
done

#Run raw2hdr with the path to the images and save as image.hdr
raw2hdr -a -e -g -f -h -w -o "$HDR_IMG/image_$DATE.hdr" "$IMG_DIR"/*dng

#Get info about the final HDR
getinfo "$HDR_IMG/image_$DATE.hdr"

#Nullify exposure value
ra_xyze -r -o "$HDR_IMG/image_$DATE.hdr" > "$OUTPUT_DIR/image_nullEV_$DATE.hd

#Resize image
pfilt -1 -x 1000 -y 1000 "$OUTPUT_DIR/image_nullEV_$DATE.hdr" > "$OUTPUT_DIR/

#Make photometric adjustment
pcomb -s 1.8 "$OUTPUT_DIR/image_resize_$DATE.hdr" > "$OUTPUT_DIR/image_photom

#Change view angle for fisheye
getinfo -a "VIEW= -vta -vv 160 -vh 160" < "$OUTPUT_DIR/image_photometric_$DAT
```

```
#Print illuminance value
echo "Total illuminance is: "
evalglare -V "$IMG_DIR/image_final_$DATE.hdr"
evalglare -V "$IMG_DIR/image_final_$DATE.hdr" > "$OUTPUT_DIR/illuminance_$DATE.txt"

#Move final HDR image to HDR_images
mv "$IMG_DIR/image_final_$DATE.hdr" "$HDR_IMG/image_final_$DATE.hdr"

#Crop to a 160
pcomb -e 'Cx:xmax/2;Cy:ymax/2;R:444.44;sq(x):x*x' -e 'inC=sq(R)-sq(x-Cx)-sq(y-Cy)' -e 'ro=if

#Make the luminance map with configurations and save it
falsecolor -s 5000 -d 1 -i "$HDR_IMG/image_crop_$DATE.hdr" --log 3 > "$MAPS_DIR/image_map_$D
```

3.2.2.3 Micrófono ambiental USB

El micrófono incorpora un filtro que atenúa eficazmente los ruidos producidos por el viento o la respiración. Su patrón de captación direccional optimiza la focalización de la fuente sonora. Steren (s. f.)

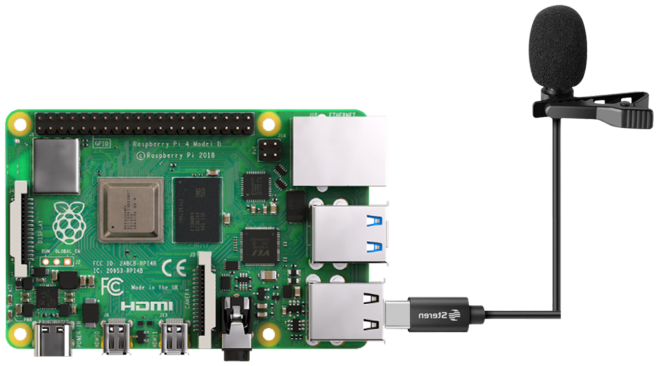

Figura 3.6: Diagrama de conexión para el micrófono.

Especificaciones

- Respuesta en frecuencia: 50 a 10,000 Hz
- Impedancia 2,200 Ω
- Sensibilidad: -32 dB ±3 dB

A continuación, se presentan los códigos utilizados para la captura y análisis de sonido. Estos scripts permiten grabar el audio, extraer parámetros clave de la señal y convertirlos a valores en decibeles (dB).

El siguiente script graba un archivo de audio en formato WAV durante 15 segundos a una frecuencia de muestreo de 44.1 kHz:

```
#!/bin/bash

grabar="/home/hdeza/Sonido/audio.wav"

arecord -D plughw:3,0 -f cd -t wav -d 15 -r 44100 $grabar
```

Posteriormente este script analiza el archivo de audio y extrae la amplitud máxima de la señal, almacenándola en un archivo de texto:

```
#!/bin/bash

maximo="/home/hdeza/Sonido/dBmax.txt"

/usr/bin/sox /home/hdeza/Sonido/audio.wav -n stat 2>&1 | /usr/bin/grep "Maxi
```

Similar al anterior, este script obtiene la amplitud mínima del audio y la guarda en un archivo de texto:

```
#!/bin/bash

minimo="/home/hdeza/Sonido/dBmin.txt"

/usr/bin/sox /home/hdeza/Sonido/audio.wav -n stat 2>&1 | /usr/bin/grep "Minimum amplitude:"
```

Este código calcula la amplitud RMS (Root Mean Square) del audio, un indicador de la potencia promedio de la señal sonora:

```
#!/bin/bash

media="/home/hdeza/Sonido/rms.txt"

/usr/bin/sox /home/hdeza/Sonido/audio.wav -n stat 2>&1 | /usr/bin/grep "RMS     amplitude:"
```

El código en Python lee los valores extraídos previamente (RMS, amplitud máxima y mínima) y los convierte a decibeles utilizando la fórmula:

$$dB = 20 \log_{10}(\text{amplitud}) + 120$$

Los valores resultantes son almacenados en un diccionario para su posterior uso:

```
import numpy as np

def sound_data():
    """
    Función para obtener los datos de sonido desde archivos y calcular sus valores en dB.

    Retorna:
        dict: Datos de sonido en dB (RMS_dB, dBmax_dB, dBmin_dB).
    """
```

```
    archivos = [
        '/home/dthisc/dthis-c/sound/rms.txt',
        '/home/dthisc/dthis-c/sound/dBmax.txt',
        '/home/dthisc/dthis-c/sound/dBmin.txt'
    ]

    rms_x = ""
    dBmax_x = ""
    dBmin_x = ""

    for archivo in archivos:
        with open(archivo, 'r', encoding='utf-8') as f:
            if archivo == '/home/dthisc/dthis-c/sound/rms.txt':
                rms_x = round(float(f.read().strip()), 4)
            elif archivo == '/home/dthisc/dthis-c/sound/dBmax.txt':
                dBmax_x = round(float(f.read().strip()), 4)
            elif archivo == '/home/dthisc/dthis-c/sound/dBmin.txt':
                dBmin_x = round(float(f.read().strip()), 4)

    rms_db = round(20 * np.log10(float(rms_x)) + 120, 4)
    dBmax_db = round(20 * np.log10(float(dBmax_x)) + 120, 4)
    dBmin_db = round(20 * np.log10(abs(float(dBmin_x))) + 120, 4)

    return {'RMS': rms_db, 'dBmax': dBmax_db, 'dBmin': dBmin_db}
```

3.3 Lista de materiales

La Tabla **??** muestra la lista de materiales necesarios para la construcción del DTHIS-C, detallando la cantidad de cada componente, los costos y la fuente de venta.

Tabla 3.4: Lista de materiales para la construcción del DTHIS-C.

Designación	Componente	Cantidad	Costo unitario (MXN)	Costo total (MXN)	Fuente de compra
Adaptador USB	UGREEN Adaptador USB C Hembra a USB Macho 3	1	$199.00	$199.00	Amazon
Arduino	Arduino UNO WiFi Rev2	1	$1,268.10	$1,268.10	AG Electrónica
Cable para cámara Raspberry	OKY9053	1	$27.59	$27.59	AG Electrónica
Carcasa	Carcasa ABS para Raspberry Pi 4 con ventilador y disipadores de calor	1	$188.00	$188.00	330ohms
Fisheye	5MP OV5647 Wide Angle Fisheye Camera	1	$502.59	$502.59	AG Electrónica

Designación	Componente	Cantidad	Costo unitario (MXN)	Costo total (MXN)	Fuente de compra
Fuente de alimentación	DC 12 V 30 A Fuente de alimentación 360 W	1	$382.00	$382.00	Amazon
Fuente para Arduino	Aclorol 9V 1A DC Fuente de alimentación	2	$269.00	$538.00	Amazon
Fuente para Raspberry	Fuente oficial para Raspberry Pi 27W USB-C	1	$370.00	$370.00	330ohms
MAX31856	3.5W Fusion SEN-30007 MAX31856	1	$1,493.10	$1,493.10	Playing With Fusion
MAX31865	Adafruit PT1000 RTD-MAX31865	1	$269.10	$269.10	Adafruit
Micro SD	Memoria Microsd Kingston Canvas Go Plus 64gb U3 V30 A2 170mb	1	$194.00	$194.00	Mercado Libre

Designación	Componente	Cantidad	Costo unitario (MXN)	Costo total (MXN)	Fuente de compra
Micrófono	Micrófono ambiental USB	1	$99.00	$99.00	Steren
Raspberry	Raspberry Pi 4 Model B 8 GB	1	$1,990.52	$1,990.52	AG Electrónica
SCD30	SCD30 Sensirion	1	$563.94	$563.94	DigiKey
Termopar	Termopar tipo T	30 m	$3,735.00	$3,735.00	Omega
TPF1	TPF1/E-20 PT1000	1	$1,576.60	$1,576.60	FuehlerSysteme
Tripié	XXZU 210 cm soporte de luz	1	$539.10	$539.10	Amazon
Wind Sensor	Wind Sensor Rev P6	1	$719.10	$719.10	Modern Device

Nota: Los precios no incluyen envío y pueden estar sujetos a cambios según el destino del producto.

4 Calibración y referenciación del DTHIS-C

En este capítulo se detalla el proceso de calibración y referenciación del DTHIS-C. Se describen las metodologías experimentales empleadas para validar su precisión, así como las pruebas realizadas en diferentes escenarios. Además, se presentan y analizan los resultados obtenidos en términos de variables térmicas, lumínicas y acústicas, ofreciendo una evaluación integral de su desempeño. Finalmente, se explican las herramientas y procedimientos para la recuperación, visualización e interpretación de los datos recopilados.

4.1 Metodología experimental

Dado que cada sensor mide diferentes variables físicas y sus mediciones se obtienen de manera distinta, se han diseñado campañas de calibración y referenciación adaptadas a las características propias de cada instrumento. Este proceso responde a la necesidad de garantizar la exactitud, confiabilidad y trazabilidad de los datos recopilados.

La metodología experimental se basa en obtener mediciones precisas que permitan validar el desempeño del dispositivo en condiciones reales. Para ello, se definieron procedimientos apoyados en instrumentos de referencia certificados, lo que facilita la detección y corrección de desviaciones inherentes a cada sensor, asegurando resultados consistentes tanto en condiciones controladas como en situaciones de campo.

La aplicación de estos procedimientos refuerza la integridad de la adquisición de datos y favorece una optimización continua del sistema, facilitando su integración en diversos contextos de aplicación.

A continuación se muestra en la Tabla **??** la lista de los sensores y sus respectivos instrumentos de referencia utilizados para la calibración y referenciación:

Tabla 4.1: Instrumentos de calibración y referenciación para los sensores que consitutyen al DTHIS-C.

Variable	**Sensor**	**Instrumento**	**Procedimiento**
Temperatura ambiente	Termopar tipo T	AMETEK Jofra PTC-155	Calibración
Temperatura radiante	TPF1/E-20 PT1000	Quest Technologies QUESTemp 32	Calibración
Velocidad del viento	Wind Sensor Rev P6	Gill Instruments WindMaster	Calibración
CO2 y humedad relativa	SCD30 Sensirion	Fluke 975 AirMeter	Referenciación
Sonido	Micrófono ambiental USB	TES-1356 Sound Level Calibrator	Calibración
Luminancia	5MP OV5647 Wide Angle Fisheye Camera		Referenciación

4.2 Obtención y visualización de los datos

4.2.0.1 ThingsBoard

ThingsBoard es una plataforma IoT de código abierto diseñada para la recopilación, procesamiento, visualización y gestión de dispositivos, ThingsBoard (2023). En el contexto de esta tesis, se utiliza para transmitir en tiempo real las mediciones obtenidas de los distintos sensores del DTHIS-C hacia un servidor central, donde se alojan y resguardan los datos en una base de datos dedicada a Internet de las Cosas.

Según la documentación oficial de ThingsBoard, la plataforma admite la conectividad mediante protocolos estándares como MQTT, CoAP y HTTP, lo que facilita la integración de diversos dispositivos y sensores, ThingsBoard (2023). En este caso, las mediciones de los sensores se envían desde el DTHIS-C a ThingsBoard, que actúa como un repositorio seguro y escalable. La plataforma no solo almacena la información, sino que también permite la creación de dashboards interactivos y personalizables que posibilitan la visualización en tiempo real de las mediciones. Esto es fundamental durante las campañas de medición, ya que permite monitorizar de forma inmediata el desempeño de los sensores, identificar tendencias y detectar posibles anomalías en los datos.

Para obtener datos directamente desde ThingsBoard, se desarrolló una función en el paquete instalable de Python denominado `iertools`, creado por el Grupo de Energía en Edificaciones. Este paquete se puede instalar desde la consola de su ordenador (previamente contando con una distribución de Python instalada) mediante el siguiente comando:

```
pip install git+https://github.com/AltamarMx/iertools.git
```

Es importante señalar que, antes de proceder a la obtención y descarga de datos desde ThingsBoard, es necesario crear un archivo de configuración con extensión `.ini`. Un archivo .ini es un documento de texto plano que

almacena parámetros y opciones en formato clave-valor, y se utiliza para definir la configuración necesaria (como la URL del servidor y el token de acceso) que el paquete `iertools` requiere para conectarse a ThingsBoard.

A continuación, se presenta un ejemplo de cómo estructurar dicho archivo:

```
[dthisc]
token     = token
device_id = device_id
tenant    = lata_testing@ier.unam.mx
password  = password
host      = tb.ier.unam.mx
port      = 8080
```

Una vez instalado `iertools`, es posible recuperar datos desde un script o notebook. El siguiente fragmento de código ilustra cómo obtener los datos de medición de los termopares del DTHIS-C a partir de ThingsBoard; este ejemplo puede ser ajustado según las necesidades específicas, permitiendo extraer distintos conjuntos de datos o modificar parámetros conforme a los requerimientos del estudio:

```
from iertools.tb import *
import warnings
warnings.filterwarnings("ignore")

fecha1 = parse("2024-11-22")
fecha2 = fecha1 + pd.Timedelta("1D")

tmp = TB(config_file='data/config.ini', device_name='dthisc')
sensores = ["TEMP1", "TEMP2", "TEMP3", "TEMP4"]

data = pd.concat([tmp.get_df(key=sensor,start_datetime=fecha1,end_datetime=fe

hora_inicio = "2024-11-22 15:42"
```

```
hora_fin = "2024-11-22 17:09"

data = data[(data.index >= hora_inicio) & (data.index <= hora_fin)]
data.sort_index(inplace=True)

data.head(10)
```

ts	TEMP1	TEMP2	TEMP3	TEMP4
2024-11-22 15:42:00.774	12.42	12.63	13.81	13.48
2024-11-22 15:42:03.505	12.37	12.57	13.81	13.46
2024-11-22 15:42:06.235	12.44	12.61	13.75	13.48
2024-11-22 15:42:08.965	12.46	12.59	13.79	13.45
2024-11-22 15:42:11.697	12.38	12.51	13.75	13.44
2024-11-22 15:42:14.427	12.35	12.56	13.77	13.41
2024-11-22 15:42:17.159	12.41	12.63	13.73	13.40
2024-11-22 15:42:19.892	12.41	12.52	13.66	13.38
2024-11-22 15:42:22.620	12.38	12.56	13.66	13.40
2024-11-22 15:42:25.351	12.34	12.52	13.72	13.34

Finalmente, para almacenar los datos recuperados, se puede utilizar el siguiente comando para exportarlos a un archivo CSV:

```
data.to_csv("ruta/de/su/preferencia.csv")
```

4.2.0.2 Temperatura ambiente

En esta campaña de calibración se utilizó el AMETEK Jofra PTC-155 (Figura **??**), un calibrador de temperatura que permite evaluar sensores en un rango de -25°C a 155°C.

Figura 4.1: AMETEK Jofra PTC-155.

El instrumento mencionado se utilizó junto a cuatro termopares tipo T, estableciendo cinco puntos de referencia de temperatura en el intervalo de 10°C a 50°C.

Para cada punto de calibración, se esperó aproximadamente 5 minutos para que el calibrador alcanzara y estabilizara la temperatura deseada. Una vez estabilizada, se registraron los datos durante 10 minutos antes de proceder al siguiente punto. Los resultados obtenidos en esta campaña se pueden observar en la Figura **??**.

`Unable to display output for mime type(s): text/html`

Figura 4.2: Rangos de temperatura utilizados para la calibración de los termopares.

4.2.0.3 Temperatura radiante

Para la medición de la temperatura radiante se empleó el QUESTemp 32 (Figura **??**), un confortímetro diseñado para evaluar el estrés térmico en el

ambiente, ya que permite medir la temperatura radiante, la temperatura del aire y la humedad relativa.

Figura 4.3: Quest Technologies QUESTemp 32.

El experimento consistió en ubicar el QUESTemp 32 dentro de una caja diseñada para limitar la incidencia directa de la radiación solar, lo que minimizó la influencia de la radiación externa y redujo el efecto de la convección forzada del viento, permitiendo así que la medición se centrara en la radiación térmica del entorno inmediato. Paralelamente, se colocó el sensor TPF1 en condiciones similares para garantizar que ambos dispositivos registraran la temperatura con la menor perturbación posible. Se tomaron datos cada minuto durante 23 horas. Los datos obtenidos se pueden observar en la Figura **??**.

`Unable to display output for mime type(s): text/html`

Figura 4.4: Intervalo de medición del QUESTemp 32 y el TPF1 para su calibración.

4.2.0.4 Velocidad del viento

El WindMaster de Gill Instruments (Figura **??**) es un anemómetro ultrasónico de alta precisión diseñado para medir la velocidad y dirección

del viento en tres direcciones (u, v, w). En esta campaña se utilizó como instrumento de referencia para la calibración del Wind Sensor.

Figura 4.5: Gill Instruments WindMaster.

La campaña de calibración tuvo una duración de 1 hora, durante la cual se capturaron datos a una frecuencia de 1 muestra por segundo. Aprovechando las dimensiones del WindSensor (2230 x 680 mm), se posicionó estratégicamente en la parte superior central del WindMaster, previa verificación de que su presencia no interfiriera con las mediciones del anemómetro. En un entorno abierto, el WindSensor registró datos de voltaje, los cuales se utilizaron junto con los datos de velocidad del viento obtenidos del WindMaster para desarrollar una ecuación de calibración que permitiera convertir el voltaje medido a m/s. Los resultados de la campaña se pueden visualizar en la Figura **??**.

`Unable to display output for mime type(s): text/html`

Figura 4.6: Comparación de la velocidad del viento registrada por el WindMaster y el voltaje del Wind Sensor para su calibración.

4.2.0.5 Humedad relativa y CO2

Para la campaña de referenciación del SCD30, el instrumento utilizado fue el Fluke 975 AirMeter (Figura **??**), este es un instrumento portátil

de diagnóstico de la calidad del aire interior. Este dispositivo mide simultáneamente temperatura, humedad relativa, dióxido de carbono (CO2) y monóxido de carbono (CO).

Figura 4.7: Fluke 975 AirMeter.

La campaña consistió en realizar mediciones simultáneas del Fluke 975 AirMeter y del SCD30 durante un período total de 7 horas y 45 minutos, registrando una muestra cada minuto. Para minimizar posibles interferencias, ambos dispositivos se ubicaron lo más próximos posible y se instalaron en una zona aislada de un espacio cerrado, de modo que la presencia humana no afectara los resultados. Los datos obtenidos se muestran en la Figura **??**.

Figura 4.9: TES-1356 Sound Level Calibrator.

4.2.0.6 Sonido

4.3 Análisis e interpretación de resultados

4.3.0.1 Temperatura ambiente

Para la calibración de los termopares, se utilizó Python junto con la librería `scikit-learn`, una herramienta utilizada para el análisis y modelado de datos. En particular, se aplicó un ajuste por regresión lineal a los datos obtenidos durante la campaña, utilizando los siguientes rangos de tiempo y sus respectivas temperaturas de referencia (Tabla **??**).

Tabla 4.3: Temperaturas de referencia y rangos de tiempo.

Temperatura (°C)	Inicio	Final
10	15:44:01	15:52:31
20	16:02:06	16:11:52
30	16:20:44	16:28:58
40	16:39:23	16:49:57
50	16:58:24	17:08:25

El ajuste por regresión lineal es una técnica estadística utilizada para modelar la relación entre una variable dependiente y una o más variables

independientes. En este caso, la variable dependiente es la temperatura de referencia (T_{ref}), mientras que las variables independientes corresponden a las lecturas obtenidas por cada termopar. El modelo lineal (Ecuación **??**) se expresa de la forma:

$$T_{ref} = m \cdot T_{sensor} + b \tag{4.1}$$

donde m es la pendiente y b es la ordenada al origen. La regresión lineal permite minimizar el error entre las mediciones del sensor y los valores de referencia, obteniendo ecuaciones de calibración (Ecuación **??**) que corrigen las lecturas de cada termopar.

$$\begin{aligned} T_{ref} &= 1.0450 \cdot TEMP1 - 2.4327 \\ T_{ref} &= 1.0602 \cdot TEMP2 - 2.8632 \\ T_{ref} &= 1.0814 \cdot TEMP3 - 4.4316 \\ T_{ref} &= 1.0554 \cdot TEMP4 - 3.9348 \end{aligned} \tag{4.2}$$

Al aplicar estas ecuaciones a los datos obtenidos en la campaña, se obtienen los valores corregidos de temperatura para cada termopar. Los resultados de la calibración pueden observarse en la Figura **??**.

`Unable to display output for mime type(s): text/html`

Figura 4.10: Resultados de las mediciones de temperatura después de aplicar la ecuación de calibración.

4.3.0.2 Temperatura radiante

Para la calibración del TPF1/E-20, se aplicó un ajuste por regresión lineal utilizando la librería `scikit-learn` de Python, a partir de los datos de temperatura registrados simultáneamente por el QUESTemp 32 y el sensor

TPF1/E-20. Empleando el mismo modelo lineal descrito en la Ecuación **??**, se derivó la siguiente ecuación de calibración (Ecuación **??**):

$$T_{ref} = 1.0582 \cdot temp - 1.5553 \tag{4.3}$$

Al aplicar esta ecuación a los datos obtenidos durante la campaña, se calcularon los valores corregidos para el TPF1/E-20. Los resultados de esta calibración se pueden observar en la Figura **??**.

`Unable to display output for mime type(s): text/html`

Figura 4.11: Comparación de las mediciones del QUESTemp 32, el TPF1/E-20 y sus datos tras la calibración.

En la tabla Tabla **??** se observa que, antes de calibrar, el sensor TPF1/E-20 presentaba un error medio de -0.1124, lo que indica un sesgo sistemático que tendía a subestimar la temperatura en comparación con el QUESTemp 32. Tras la aplicación de la ecuación de calibración, el error medio se reduce drásticamente a 4.8948e-15, un valor prácticamente nulo que evidencia la eliminación casi total del sesgo sistemático. Paralelamente, el error medio absoluto disminuye de 0.1518 a 0.0344, lo que implica una mejora significativa en la precisión global de las mediciones. Estos resultados confirman que el proceso de calibración ha corregido eficazmente tanto el sesgo como la dispersión de los datos, permitiendo obtener mediciones de temperatura de alta fiabilidad y precisión, estrechamente alineadas con los valores de referencia del QUESTemp 32.

Tabla 4.4: Error medio y error medio absoluto antes y después de la calibración.

Fase	Error medio	Error medio absoluto
Antes de calibrar	-0.1124	0.1518
Después de calibrar	4.8948e-15	0.0344

4.3.0.3 Velocidad del viento

Durante la calibración del Wind Sensor se siguió la metodología proporcionada por el fabricante Device (2017) en su blog oficial. Según esta fuente, la relación entre la velocidad del viento (WS), el voltaje corregido y la temperatura sigue una ley de potencia descrita por la siguiente ecuación (Ecuación **??**):

$$WS = a \cdot \text{voltaje}^b \cdot \text{temp}^c \tag{4.4}$$

Adicionalmente, se identificó un error sistemático en las lecturas de voltaje: en condiciones de viento nulo (0 m/s), el Wind Sensor registraba un desfase de 1.1621 V. Este error se corrigió restando 1.1621 V a cada medición, de modo que el voltaje empleado en el análisis reflejara únicamente la contribución del viento.

Para facilitar el ajuste mediante técnicas de regresión, se aplicó una transformación logarítmica a ambos lados de la ecuación, lo que permitió convertir el modelo en una forma lineal (Ecuación **??**):

$$\ln(WS) = \ln(a) + b \cdot \ln(\text{voltaje}) + c \cdot \ln(\text{temp}) \tag{4.5}$$

Esta transformación es crucial, ya que permite emplear técnicas de regresión lineal para estimar los parámetros $ln(a)$, b y c a partir de los datos experimentales. Una vez obtenidos, se recupera el parámetro a mediante la función exponencial ($a = exp(ln(a))$) y se formula la ecuación de calibración en su forma original. En este dominio lineal, se utilizó la librería `scikit-learn` de Python para estimar los parámetros del modelo.

Adicionalmente, se identificó un error sistemático en las lecturas de voltaje: en condiciones de viento nulo (0 m/s), el Wind Sensor registraba un desfase de 1.1621 V. Este error se corrigió restando 1.1621 V a cada medición, de modo que el voltaje empleado en el análisis reflejara únicamente la contribución del viento.

Para el caso elegido, correspondiente a un remuestreo de 10 segundos, el proceso de regresión lineal en el dominio logarítmico produjo la siguiente ecuación de calibración (Ecuación **??**):

$$WS = 26.3431 \cdot \text{voltage}^{1.4273} \cdot \text{temp}^{-0.7631} \tag{4.6}$$

Esta ecuación se emplea para convertir las mediciones de voltaje (tras aplicar la corrección) y de temperatura en valores precisos de velocidad del viento (m/s).

El coeficiente de correlación obtenido tras aplicar el modelo es un indicador de la calidad del ajuste. Valores de correlación cercanos a 1 implican una excelente concordancia entre las mediciones del Wind Sensor (tras la calibración) y las obtenidas con el WindMaster, lo que confirma que el modelo captura de manera adecuada la variabilidad de los datos. En la Tabla **??** se resumen los coeficientes de correlación para los diferentes intervalos de remuestreo.

Tabla 4.5: Coeficientes de correlación para los distinto remuestreos.

Remuestreo	Coeficiente de correlación
3 segundos	0.7622
5 segundos	0.8079
10 segundos	0.8637

El aumento progresivo en el coeficiente de correlación, alcanzando 0.8637 para un remuestreo de 10 segundos, indica que el modelo de calibración y la corrección del desfase mejoran significativamente la consistencia de las mediciones del Wind Sensor con respecto al WindMaster. Estos hallazgos validan la eficacia del procedimiento de calibración y pueden observarse en la Figura **??**.

```
Unable to display output for mime type(s): text/html
```

Figura 4.12: Comparación de las mediciones de velocidad del viento registradas por el WindMaster y el Wind Sensor tras su calibración.

4.3.0.4 Humedad relativa y CO2

Para esta campaña de referenciación, se compararon directamente los datos de medición obtenidos por el SCD30 y el Fluke 975 AirMeter, dado que el SCD30 se calibra de forma independiente. Se realizó el cálculo de errores, en particular el error medio y el error medio absoluto, para evaluar la discrepancia entre las mediciones de ambos instrumentos. Estos errores permiten determinar si existe algún sesgo sistemático en las lecturas del SCD30 respecto al Fluke.

El error medio se obtiene al calcular la diferencia promedio entre las mediciones del SCD30 y las del Fluke. Un valor negativo indica que, en promedio, el SCD30 tiende a subestimar la variable, mientras que un valor positivo indica una sobreestimación. El error medio absoluto cuantifica la magnitud promedio del error sin considerar el signo, ofreciendo una medida de la precisión global de las mediciones.

La Tabla **??** resume los resultados obtenidos para cada variable:

Tabla 4.6: Error medio y error medio absoluto para cada variable del SCD30 con respecto al Fluke.

Variable	Error medio	Error medio absoluto
Humedad relativa	-2.1681	2.1681
CO2	9.6543	13.1565
Temperatura	0.8853	0.8853

La tabla indica que, para la humedad relativa, el SCD30 subestima en

promedio 2.1681 unidades respecto al Fluke, mientras que para la concentración de CO2 se observa una sobreestimación de 9.6543 unidades en promedio, con un error medio absoluto mayor que su error medio, lo que sugiere una mayor dispersión en las mediciones. En el caso de la temperatura, la discrepancia es menor, con un error medio de 0.8853.

La correlación entre las mediciones de ambos instrumentos se visualiza en la Figura **??**, lo que respalda la evaluación de precisión y consistencia. Un coeficiente de correlación elevado indicaría que, a pesar de los errores sistemáticos, la tendencia de las mediciones del SCD30 sigue de cerca la del Fluke.

`Unable to display output for mime type(s): text/html`

Figura 4.13: Correlación de las mediciones de humedad relativa, CO2 y temperatura entre el Fluke y el SCD30.

4.3.0.5 Sonido

5 Discusión y conclusiones

5.1 Trabajo a futuro

Bibliografía

Adafruit. s. f. «Adafruit PT1000 RTD Temperature Sensor Amplifier - MAX31865». https://www.adafruit.com/product/3648#description.

Antoniadou, Panagiota, y Agis M. Papadopoulos. 2017. «Occupants' thermal comfort: State of the art and the prospects of personalized assessment in office buildings». *Energy and Buildings* 153: 136-49. https://doi.org/10.1016/j.enbuild.2017.08.001.

Arducam. s. f. «Arducam 5MP OV5647 Ultra Wide Angle Fisheye Camera for Raspberry Pi». https://www.arducam.com/product/arducam-ultra-wide-angle-fisheye-5mp-ov5647-camera-for-raspberry-pi/.

Arduino. 2023. «Arduino UNO WiFi Rev2». https://store.arduino.cc/products/arduino-uno-wifi-rev2.

ASHRAE-55. 2021. *ANSI/ASHRAE Standard 55-2020: Thermal Environmental Conditions for Human Occupancy*. Atlanta, GA: American Society of Heating, Refrigerating; Air-Conditioning Engineers (ASHRAE).

ASHRAE-62. 2001. *ANSI/ASHRAE Standard 62-2001: Ventilation for acceptable indoor air quality*. Vol. 62. 2001. American Society of Heating, Refrigerating; Air-Conditioning Engineers.

Azar, Elie, William O'Brien, Salvatore Carlucci, Tianzhen Hong, Andrew Sonta, Joyce Kim, Maedot S. Andargie, et al. 2020. «Simulation-aided occupant-centric building design: A critical review of tools, methods, and applications». *Energy and Buildings* 224: 110292. https://doi.org/10.1016/j.enbuild.2020.110292.

Berglund, B. 1999. «Guidelines for community noise». *World Health Organization*.

CIE. 2020a. «Illuminance (17-21-060)». https://cie.co.at/eilvterm/17-21-060.

———. 2020b. «Luminance (17-21-050)». https://cie.co.at/eilvterm/17-21-050.

Device, Modern. 2017. «Calibrating The Rev. P Wind Sensor From A New Regression». https://moderndevice.com/blogs/documentation/calibrating-the-rev-p-wind-sensor-from-a-new-regression.

Djongyang, Noël, René Tchinda, y Donatien Njomo. 2010. «Thermal comfort: A review paper». *Renewable and Sustainable Energy Reviews* 14 (9): 2626-40. https://doi.org/10.1016/j.rser.2010.07.040.

FuehlerSysteme. s. f. «Radiation Pendulum Temperature Sensor». https://www.fuehlersysteme.de/Radiation-Pendulum-Temperature-Sensor_1#70bd/embedded/.

Macpherson, R. K. 1962. «The assessment of the thermal environment: A review». *British Journal of Industrial Medicine* 19 (3): 151-64. https://doi.org/10.1136/oem.19.3.151.

ModernDevice. s. f. «Wind Sensor Rev. P». https://moderndevice.com/products/wind-sensor-rev-p.

OMEGA. s. f. «¿Qué es un sensor termopar?» https://es.omega.com/prodinfo/termopares.html#.

PWFusion. s. f. «4-Channel T-Type Thermocouple MAX31856 SPI Digital Arduino Shield». https://www.playingwithfusion.com/productview.php?pdid=71&catid=1008.

Raspberry. 2019. «Raspberry Pi 4 Model B». https://www.raspberrypi.com/products/raspberry-pi-4-model-b/.

Sensirion. s. f. «SCD30». https://sensirion.com/products/catalog/SCD30.

Standard, British. 2011. «Light and lighting: Lighting of work places: Part 1: Indoor work places». *NS-EN 12464-1: 2011*.

Steren. s. f. «Micrófono USB C de solapa para celular». https://www.steren.com.mx/microfono-usb-c-de-solapa-para-celular.html.

ThingsBoard. 2023. «ThingsBoard: Open-source IoT platform for data collection, processing, visualization, and device management». https://thingsboard.io/.

Vásquez, Natalia Giraldo, Maíra Longhinotti Felippe, Fernando O. R.

Pereira, y Ariane Kuhnen. 2019. «Luminous and visual preferences of young children in their classrooms: Curtain use, artificial lighting and window views». *Building and Environment* 152: 59-73. https://doi.org/10.1016/j.buildenv.2019.01.049.

Anexos

This is a book created from markdown and executable code.